Orientation et tri multimodal des demandeurs d'emploi par l'Intelligence Artificielle

For external use

14/10/2026 à 12h – Romain Busuttil

Dans le cadre la certification Atlas :
« Concevoir et implémenter une solution d'intelligence artificielle (IA) »

Atos

1. Cadrage métier & problème
2. Sources des données & EDA
3. Modélisation & entrainement
4. Industrialisation
5. Suivi en production & amélioration continue
6. Conclusion et questions ouvertes

01

Cadrage métier & problème

Cadrage métier & problème

Quel est le problème ? Comment le résoudre ?



Contexte client

- Agence nationale de l'emploi / service public
- Problème métier : orienter des demandeurs d'emploi

Tâche IA

- Prédire le délai de retour à l'emploi en 3 classes (0 : rapide <6 mois, 1 : moyen 6-12 mois, 2 : long >12 mois)
- A partir de données multimodales (tabulaire + texte libre)



Impacts

- Direct : aide à l'orientation pour les conseillers
- Indirect : les demandeurs d'emploi

Cadrage métier & problème - Réglementaires anticipés

Quels risques éthiques et réglementaires entraine l'utilisation de l'IA dans ce contexte ?

RGPD

- Pseudonymisation
- Droit à l'oubli
- Minimisation des données
- Analyse d'impact



AI Act

- Supervision humaine
- Documentation
- Robustesse
- Gestion des logs
- Suivi des dérives



Droit sectoriel

- Non-discrimination algorithmique
- Information sur l'utilisation de l'IA
- Expliquer la logique de traitement



Responsabilité juridique

- Responsabilité en cas d'erreur?
- Tracer la recommandation et la décision



02

Sources des données & Exploration et analyse des données

Sources des données

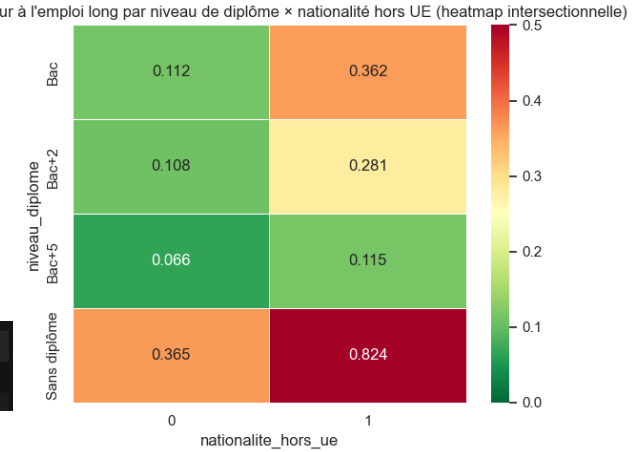
Quelle est la qualité des données et comment se répartissent-elles dans le jeu de données ?

- Fichier CSV fourni dans le cadre de l'examen CISIA : dataset de 2500 lignes et 10 colonnes

Variables	Type	Sensible/biais
usager_id	ID	✅
age	Numérique	✅
niveau_diplome	Catégoriel	❌
anciennete_poste_ans	Numérique	❌
code_rome_vise	Catégoriel	❌
code_insee_commune	Catégoriel	✅
est_allocataire	Binaire	✅
nationalite_hors_ue	Binaire	✅
synthese_entretien	Texte	?
classe_retour_emploi	Catégoriel	

- Points clés de l'EDA :
  - Pas de doublon
  - Valeurs manquantes
  - Nombre limité de synthèse d'entretien
  - Certaines features déséquilibrées
  - Cible déséquilibrée
  - Effet intersectionnel
  - Disparate Impact bas

	usager_id	age	niveau_diplome	anciennete_poste_ans	classe_retour_emploi
89	ID_0089	18.0	Bac	7.0	1
94	ID_0094	18.0	Bac	5.2	0



Exploration & analyse des données (EDA)

Quelle est la qualité des données et comment se répartissent-elles dans le jeu de données ?

- Conclusion et révision des critères de succès

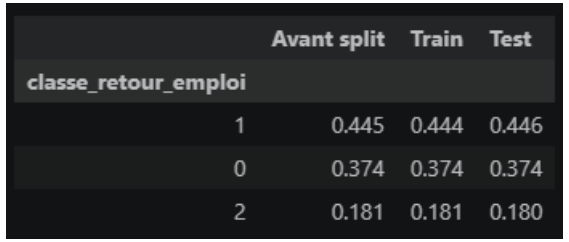
Type	Critère	Cible initiale (client)	Cible révisée après EDA	Justification de la révision
Métier	Réduire les erreurs critiques (classe 2 prédit 0)	<5%	<5%	On va essayer de minimiser l'erreur critque qui a le plus fort coût humain et financier. Cependant la classe 2 est minoritaire et il y a peu de données, cela sera dur à prédire donc on reste sur la même cible.
Modèle	F1-score macro	F1-score macro ≥ 0.75	F1-score macro ≥ 0.70	La classe 2 est minoritaire et il y a peu de données, une revue à la baisse est nécessaire.
Modèle	Matrice de confusion	Recall ≥ 85% pour classe 2	Recall ≥ 70% pour classe 2	La classe 2 est minoritaire et il y a peu de données, une revue à la baisse est nécessaire.
Opérationnel	Temps de réponse API en production	Latence : P50 < 250 ms	on ne modifie pas	ça reste réaliste
Robustesse	Traiter les manquants et les incohérences	non défini précédemment	100%	Il faut que les manquants soient traités à 100%
Equité	Limiter les disparités entre sous-groupes sensibles	non défini précédemment	écart de recall classe 2 entre groupes ≤ 10	L'EDA montre des DI < 0,8 et donc un risque de discrimination statistique

Préparation des données et scénarios de jeux de données

Comment préparer les données de manière fiable et reproductible ?

- Prétraitement :
  - Remplacement du code INSEE par le département
  - Incohérences age <-> ancienneté : on positionne ancienneté à *NaN*
  - Exclue les identifiants *usager_id*
- Split des données :

Jeu	Lignes	Part	Classe 0 / 1 / 2
Train	2 000	80 %	749 / 889 / 362
Test	500	20 %	187 / 223 / 90
Protocole	Stratifié, random_state fixé	-	Répartition concervé



classe_retour_emploi	Avant split	Train	Test
1	0.445	0.444	0.446
0	0.374	0.374	0.374
2	0.181	0.181	0.180

- **Pipelines** :

Type de variable	Traitements
Numérique	Valeurs manquantes = médiane + mise à l'échelle
Catégoriel (ordinal)	Manquantes = modalité la plus fréquente + encode selon l'ordre
Catégoriel (nominal)	Manquantes = modalité la plus fréquente + encodage one-hot + ignore les modalités inconnues
Texte	Manquantes = vide + Transforme 2D en 1D + Vectorisation TF-IDF

03

Modélisation & entrainement

Préparation des données et scénarios de jeux de données

Comment préparer les données de manière fiable et reproductible ?



Scénario 1 — Approche multimodale complète

- Toutes les variables (tabulaires + texte vectorisé)



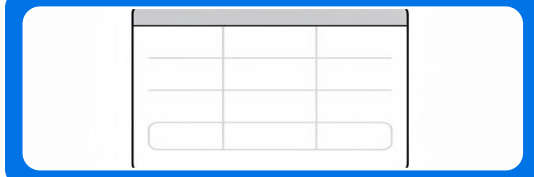
Scénario 2 — Sans variables sensibles (Approche Éthique)

- Variables non sensibles uniquement (avec justification CNIL/RGPD)



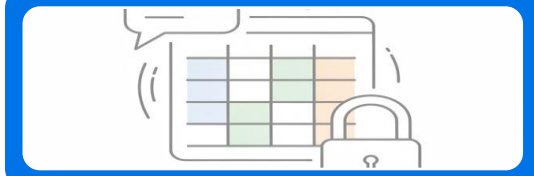
Scénario 3 — Diagnostic par le texte seul (NLP pure)

- synthese_entretien uniquement



Scénario 4 — Données contextuelles pures (Tabulaire seul)

- Variables tabulaires uniquement (sans texte)



Scénario 1 — Approche hybride (ajouté dans un second temps)

- Toutes les variables à l'exception de `nationalite_hors_ue`

Familles candidates & Modèles candidats

Quelles familles de modèles peut-on envisager pour cette classification, et quels modèles retenir ?



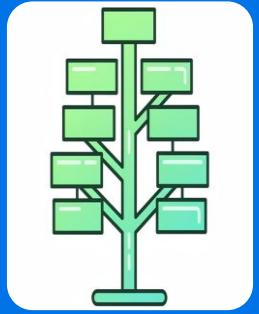
ML classique (sklearn, XGBoost)

- Mieux adapté
- Faible coût
- Forte explicabilité



LogisticRegression

- Baseline robuste, rapide et stable
- Coût entrainement : faible
- Coût inférence : faible
- Explicabilité : élevée



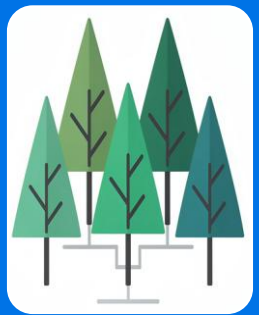
LightGBM Classifier

- Adaptée aux features encodées en grande dimension
- Coût entrainement : moyen
- Coût inférence : faible à moyen
- Explicabilité : moyenne



XGBoost Classifier

- Très bon compromis attendu sur tabulaire
- Coût entrainement : moyen à élevé
- Coût inférence : moyen
- Explicabilité : moyenne



RandomForestClassifier

- Bon point de comparaison face aux méthodes de boosting
- Coût entrainement : moyen
- Coût inférence : faible
- Explicabilité : moyenne

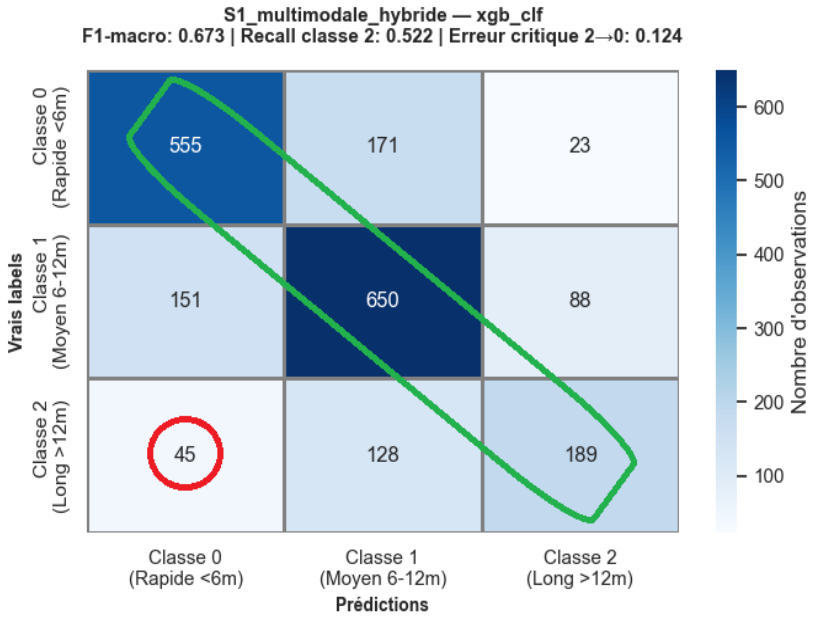
Métriques retenues & justification

Quels indicateurs choisir pour évaluer le modèle et pourquoi sont-ils adaptés à notre objectif métier ?

Métrique	Ce qu'elle mesure	Pourquoi elle est retenue ici
f1_macro	Moyenne des F1 de chaque classe (precision × recall équilibrés), pondération égale entre classes	**Mesure principale** : pénalise les modèles qui ignorent une classe. Insensible au déséquilibre car ne dépend pas des effectifs. Permet de comparer tous les candidats sur le même pied.
écart-type (f1_macro)	Dispersion du f1_macro entre les 5 folds de la validation croisée	Mesure la **stabilité** du modèle : un f1_macro élevé mais avec un écart-type fort indique un modèle instable, sensible à la composition du fold. Un modèle stable (faible écart-type) est plus fiable en production.
recall_classe_2	Parmi tous les vrais demandeurs d'emploi de classe 2 (risque de longue durée), quelle fraction le modèle détecte-t-il ?	Classe 2 = profils les plus vulnérables. Un recall faible signifie qu'on rate des personnes qui auraient besoin d'un accompagnement renforcé → coût social élevé. **Sensibilité prioritaire.**
precision_classe_2	Parmi les prédictions classe 2, quelle fraction est vraiment classe 2 ?	Évite les fausses alertes : déclencher un accompagnement renforcé inutilement est coûteux en ressources. Complète le recall pour ne pas sur-classifier.
f1_classe_2	Moyenne harmonique de recall et precision sur la seule classe 2	Synthèse équilibrée du compromis détection / faux positifs sur la classe la plus sensible. Métrique de décision complémentaire au f1_macro.
erreur_critique_2_vers_0	Taux de demandeurs d'emploi réellement classe 2 prédits comme classe 0 (pas de difficulté)	**Erreur la plus grave** du système : orienter vers "retour rapide" quelqu'un qui a besoin d'un soutien long. Cette métrique custom quantifie précisément ce risque. À minimiser absolument.
log_loss	Pénalise les probabilités mal calibrées (confiance élevée sur une mauvaise prédiction)	Mesure la qualité de calibration : un modèle qui dit "95 % classe 0" quand classe 2 est plus dangereux qu'un modèle incertain. Utile pour comparer la fiabilité des sorties de probabilité entre candidats. Le plus proche de 0 est le mieux

Validation croisée & Optimisation des hyperparamètres

Comment optimiser les hyperparamètres pour améliorer les performances des modèles ?



- Paramètres retenus :

objective="multi:softprob"

num_class=3

n_estimators=500

learning_rate=0.03

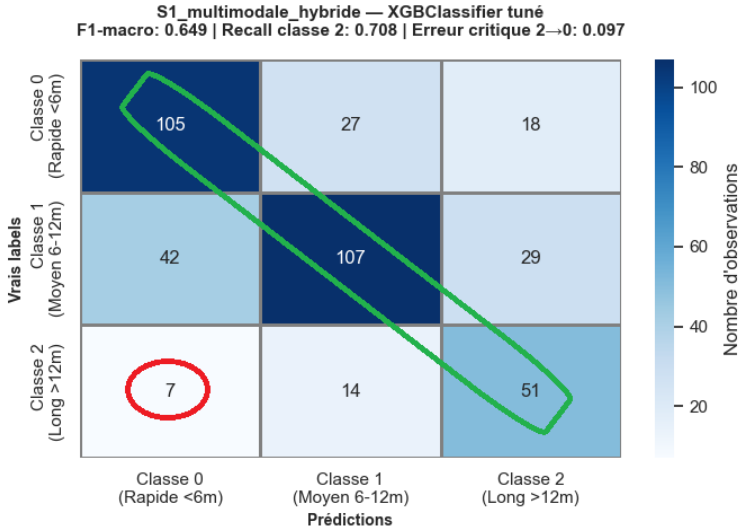
max_depth=5

colsample_bytree=0.7

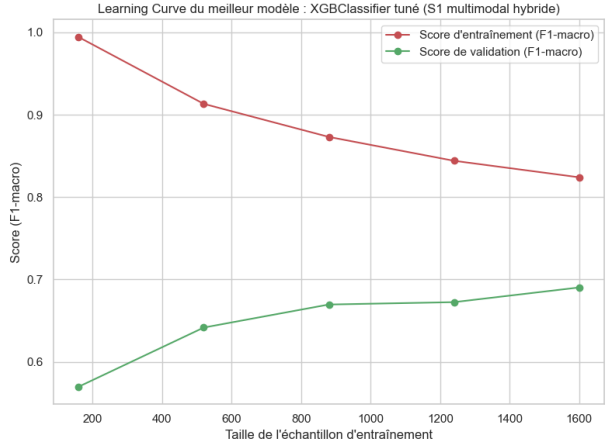
random_state=RANDOM_STATE

n_jobs=-1,

class_weights_lr = {0: 1.0, 1: 1.0, 2: 3.0}



- Courbe d'apprentissage :



- Filet de sécurité et revue humaine :

Seuil	Dossiers en revue	Erreur critique	Erreur critique en revue
0,2	127	7	3
0,25	124	7	3
0,30	124	7	3
0,35	124	7	3
0,4	124	7	3

Comparaison des finalistes

Comment optimiser les hyperparamètres pour améliorer les performances des modèles ?

- Hypothèse de coûts :

Type d'hypothèse	Coût proposé
Erreur d'orientation modérée « 0 → 1 » ou « 1 → 0 »	1 000 €
Intensité d'accompagnement mal évaluée « 1 → 2 » ou « 2 → 1 »	5 000 €
Mobilisation excessive de ressources « 0 → 1 »	8 000 €
Erreur critique « 0 → 1 »	15 000 €
Revue manuelle d'un dossier	20 €

- Résultats :

Scénario	Modèle	F1 macro	Rappel C2	Erreur critique	Coût après filet
S1 hybride	XGBoost	**0,692 (± 0,033)**	0,703	0,110	**234 270 €**
S1 hybride	LightGBM	0,627 (± 0,019)	0,703	0,124	391 100 €
S1 hybride	RF	0,597 (± 0,025)	0,807	**0,059**	371 970 €
S1 hybride	Reg Log	0,561 (± 0,017)	**0,841**	0,062	448 020 €
S2	Reg Log	0,634 (± 0,029)	0,483	0,224	265 370 €
S3	RF	0,637 (± 0,028)	0,666	0,176	300 190 €

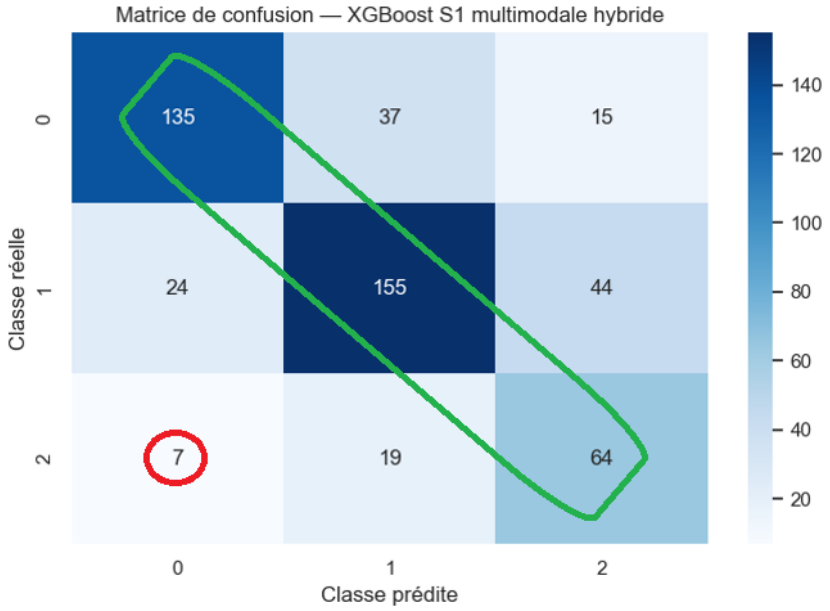
Evaluation sur le test

Le couple XGBoost & S1 hybride conserve-t-il de bonnes performances sur des données jamais vues ?

Modèle retenu	Scénario	Erreur critique	F1-macro	Recall C2	Hyperparamètres principaux
XGBClassifier pondéré	S1 multimodale hybride	0,110	0,692 ± 0,033	0,703	n_estimators=500, learning_rate=0.03, max_depth=5, colsample_bytree=0.7, poids `{0:1, 1:1, 2:3}`

- Coût d'inférence : 0,50 € par heure de CPU

- Résultat sur le test final :



F1 macro	Recall C2	Précision C2	F1 C2	Erreur critique	Latence p95	Coût estimé
0,693	0,711	0,520	0,601	0,078	10,167 ms	0,007852 €

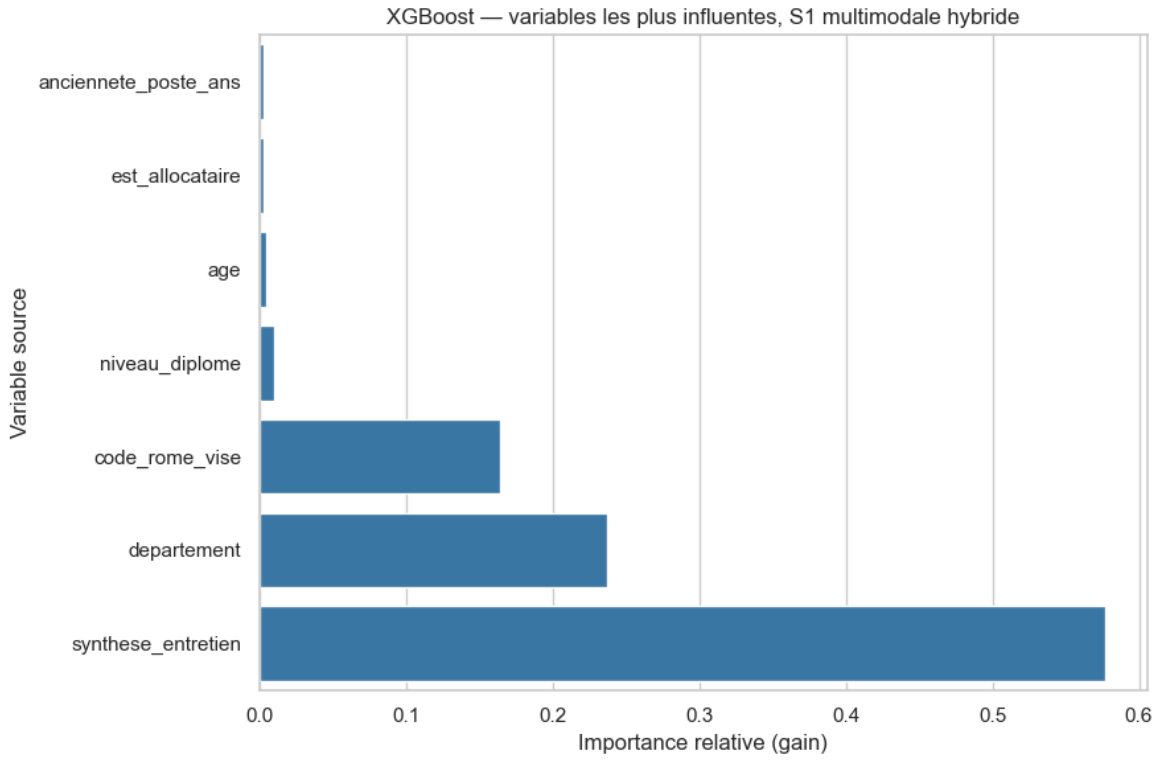
Comparaison avec la baseline DummyClassifier

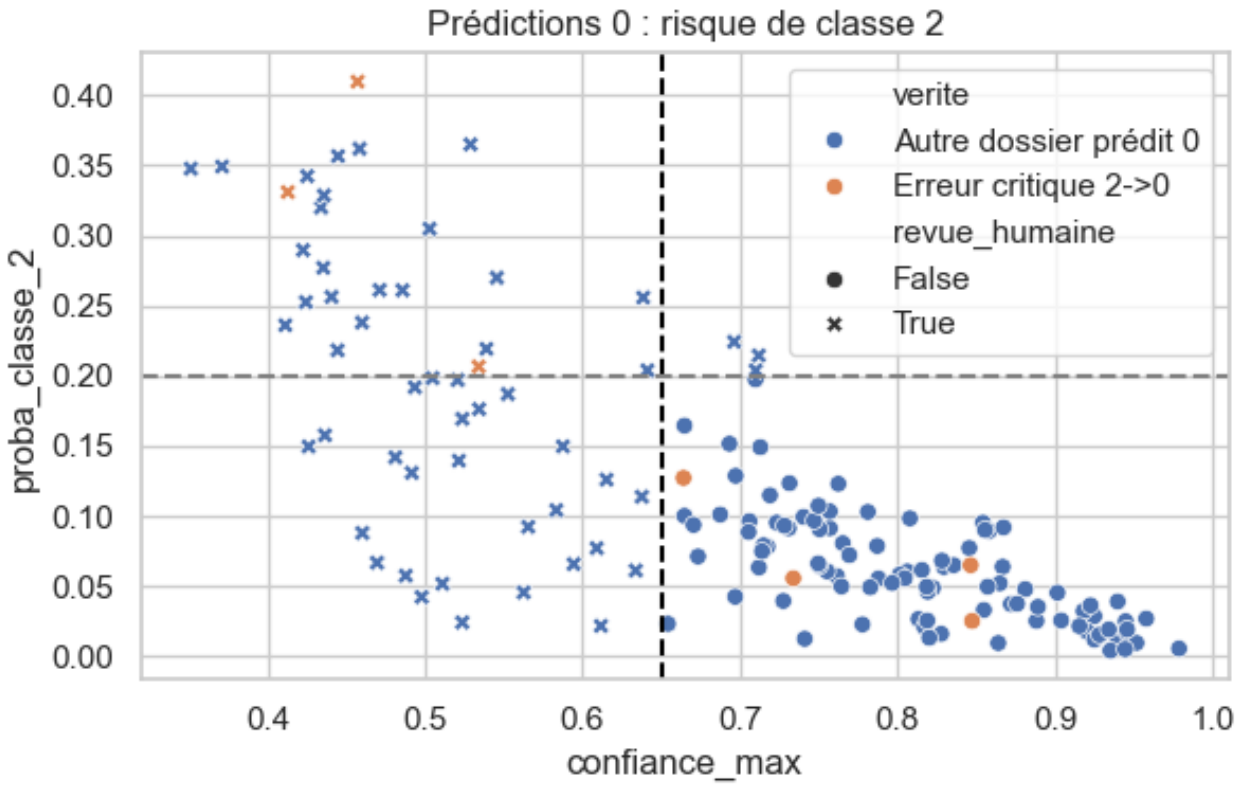
Critère	DummyClassifier	Modèle retenu
F1 macro	0,206	0,693
Recall classe 2	0,000	0,711
Erreur critique	0,000	0,078

- Limites :
  - Données limitées
  - textes entretien très redondants

Poids des variables & filet de sécurité

Comment interpréter le fonctionnement du modèle et ses limites pour la communication au client ?





Message au client

Comment interpréter le fonctionnement du modèle et ses limites pour la communication au client ?



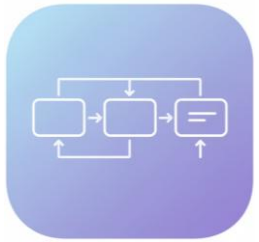
Messages clés

- Le modèle donne un repère, pas un diagnostic
- La vérification humaine est indispensable et ne repère pas tous les cas à risque
- Le conseiller garde la décision finale



Synthèse d'interprétation

- Variables-clés : synthèse d'entretien (57,7%), département (23,7%), code ROME (16,5%), âge (0,5%), statut allocataire (0,3%)
- Profils mal prédits
- Vigilance opérationnelle



Conclusion scénarios

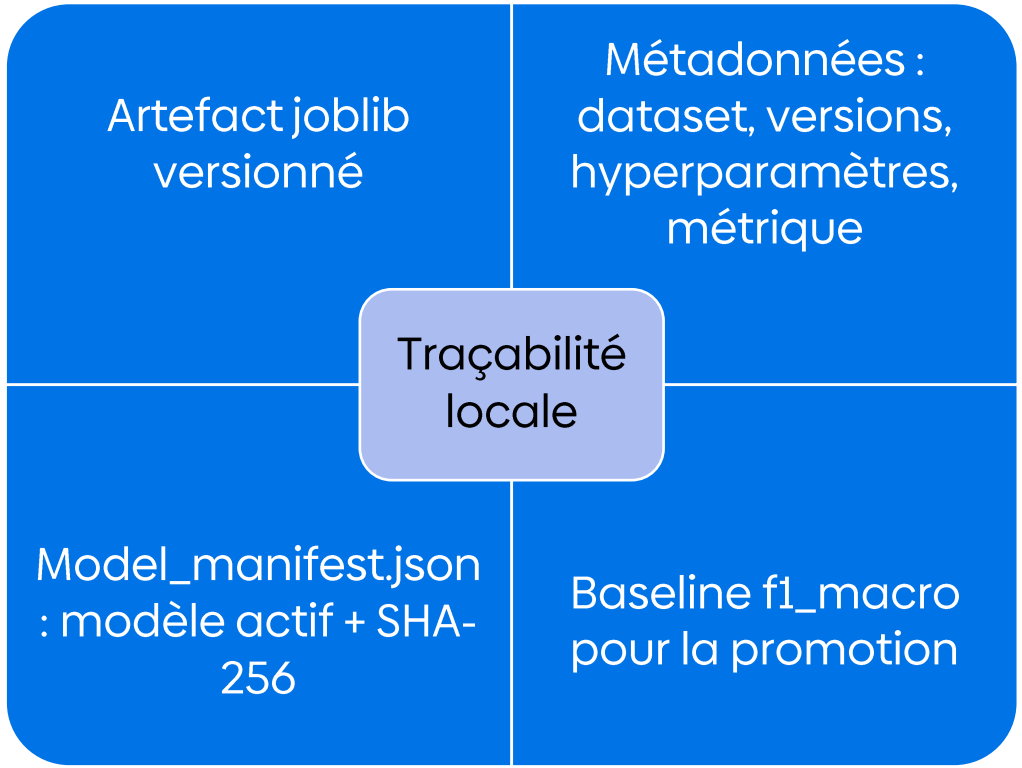
- S1 complet : la meilleur performance (mais variable discriminante)
- S2 : le moins performant
- S3 : le compromis fragile
- S4 : le moins cher (mais variable discriminante)
- S1 hybride : le meilleur compromis

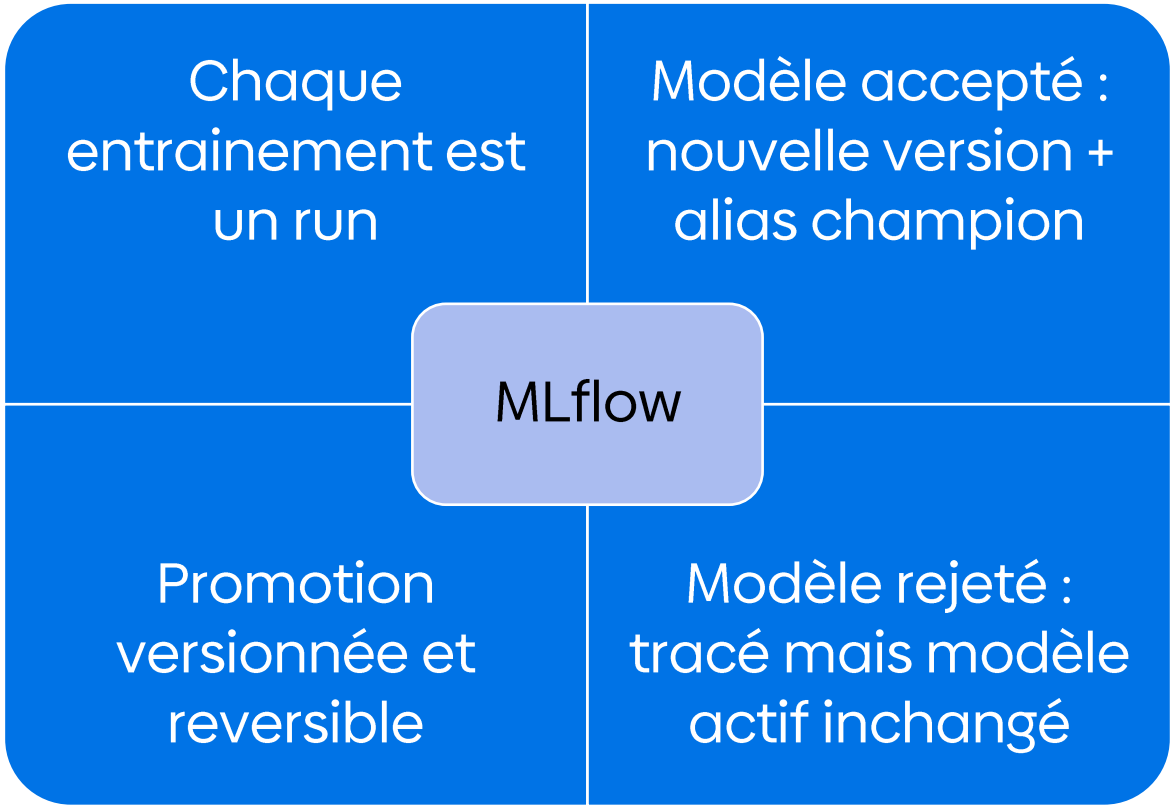
04

Industrialisation

Persistance et suivi des versions du modèle

Comment sauvegarder, tracer et versionner les modèles ?

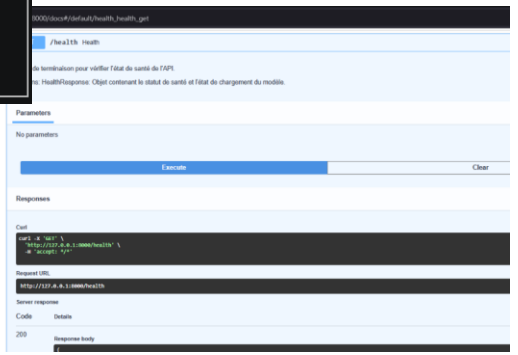




API de service

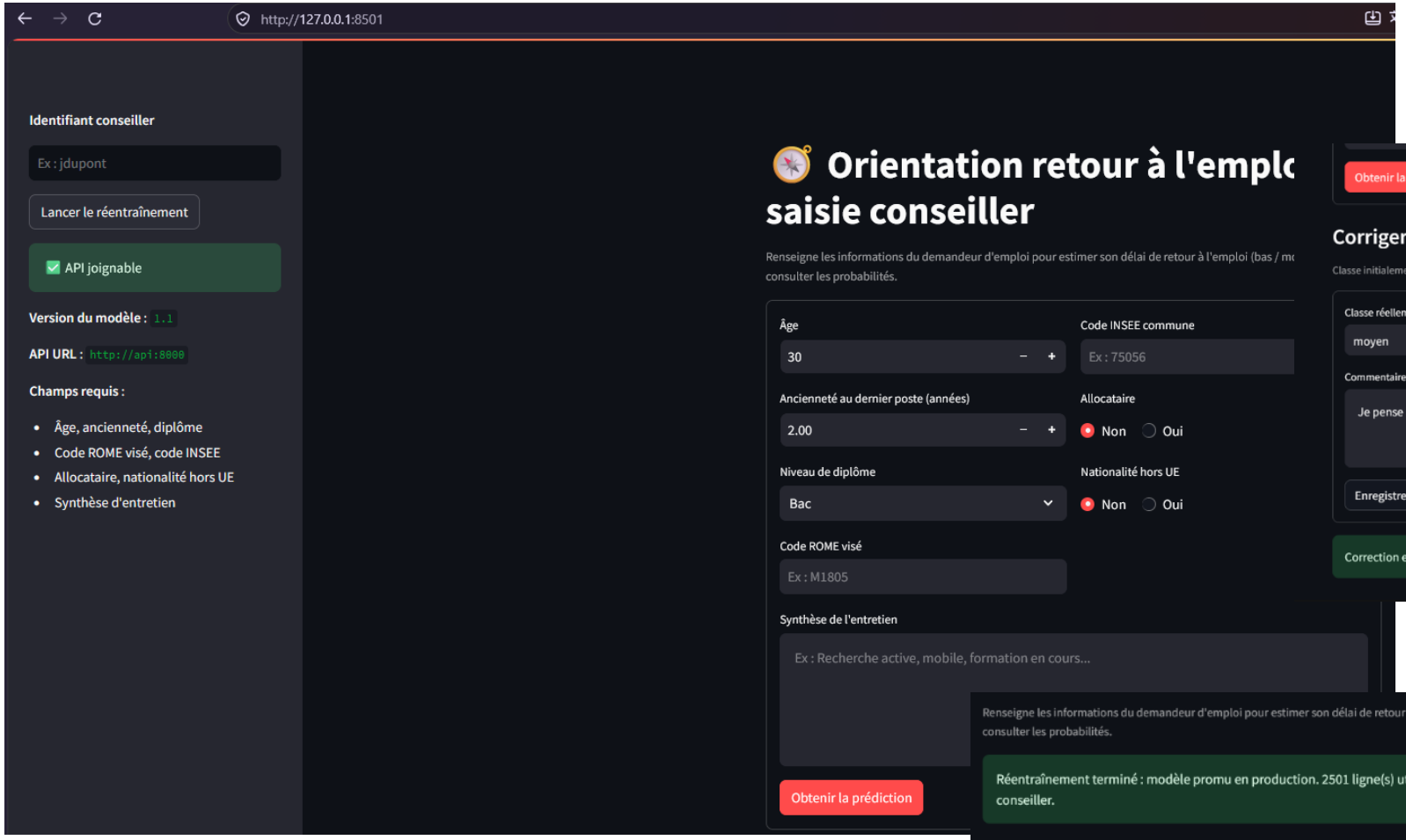
Comment exposer le modèle via une API ?

Route	Méthode	Rôle	Réponse nominale	Erreurs principales
`/health`	`GET`	Vérifier que l'API répond et que le pipeline actif, désigné par `model_manifest.json`, est chargé et intègre	`200` avec `status`, `model_loaded` et `model_version`	`status="degraded"` si le modèle est absent, invalide ou impossible à charger
`/predict`	`POST`	Prédire le délai de retour à l'emploi pour un demandeur	`200` avec classe `bas/moyen/long` et probabilités	`422` si le payload est invalide, `503` si le modèle actif est indisponible
`/feedback`	`POST`	Enregistrer la correction d'un conseiller pour le prochain réentraînement	`200` avec statut et nombre total de corrections stockées	`422` si le payload est invalide
`/train`	`POST`	Réentraîner le pipeline, intégrer les feedbacks, comparer le modèle à la baseline et tracer l'entraînement avec MLflow	`200` avec statut (`trained`/`rejected`), métriques, décision de promotion, `model_path`, `artifact_version`, `mlflow_run_id` et `model_version` si le modèle est promu	`404` si le dataset manque, `422` si l'entraînement ou le suivi MLflow échoue
`/history`	`GET`	Consulter l'historique persistant des prédictions, filtrable par conseiller et limité par `limit`	`200` avec la liste des prédictions passées, la plus récente en premier	Liste vide si aucun historique n'existe encore

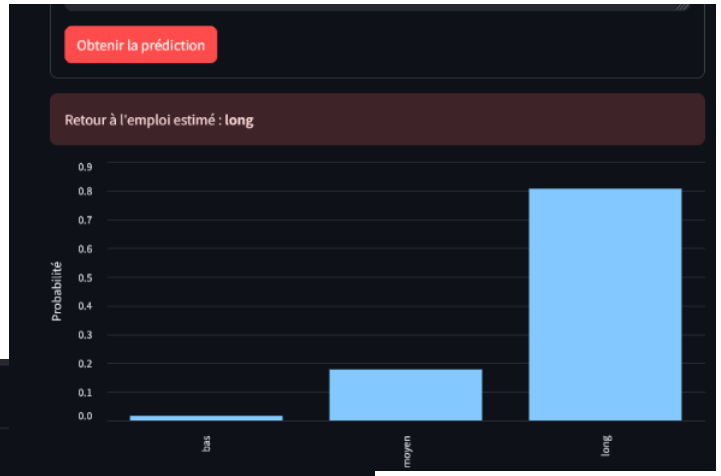


Interface utilisateur

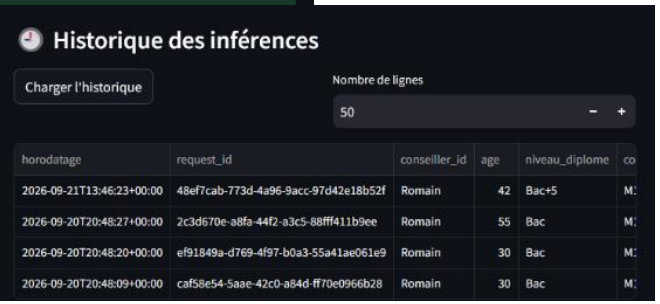
Comment permettre au conseiller d'utiliser le modèle ?





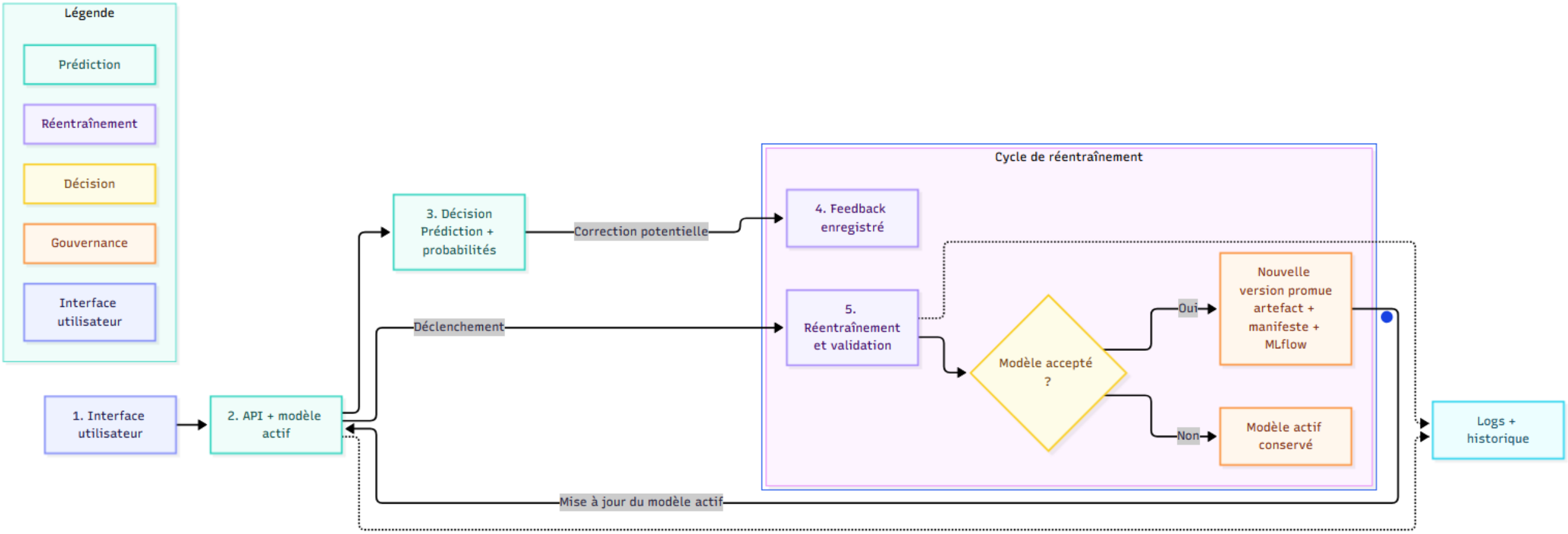






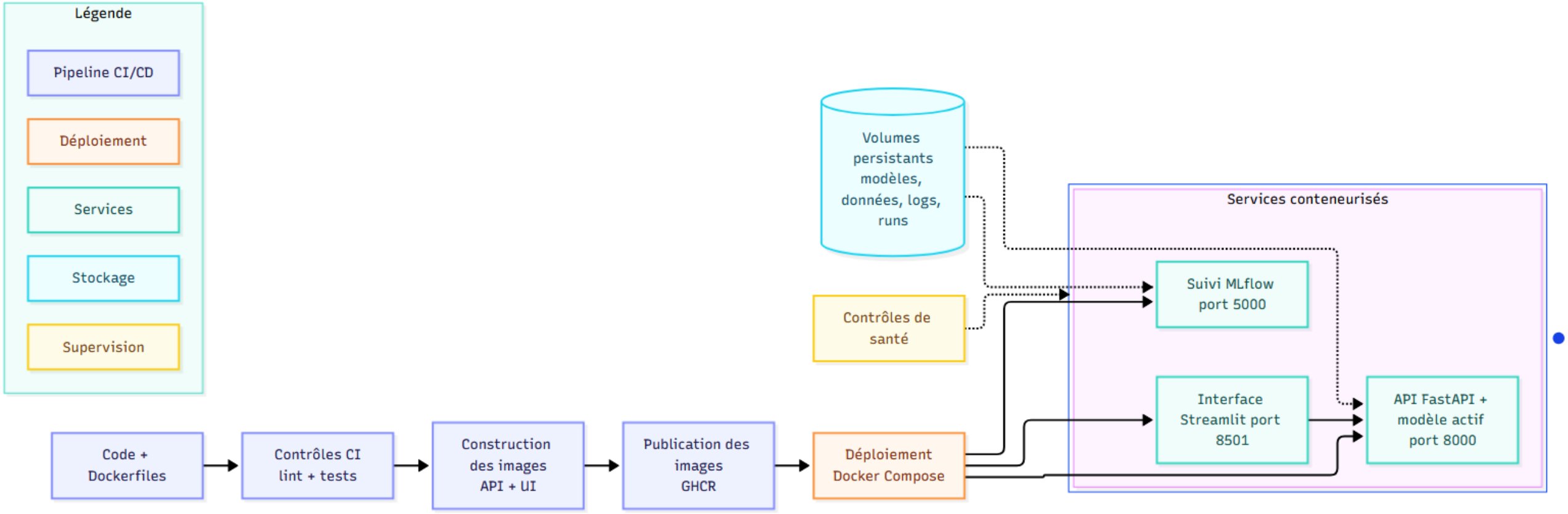
Schémas d'architecture

Comment s'articulent l'architecture et les flux de données ?



Conteneurisation & CI/CD

Comment conteneuriser la solution et automatiser son déploiement ?



Points encore ouverts

Quels prérequis restent à lever avant la mise en production ?



Garde-fou de promotion incomplet



Monitoring opérationnel incomplet



Persistance MLflow non adaptée à la production



Validation humaine de la promotion absente



Qualité des feedbacks à contrôler



Surveillance des dérives non implémentée



Procédure de rollback non validée



Authentification d'accès

05

Suivi en production & amélioration continue

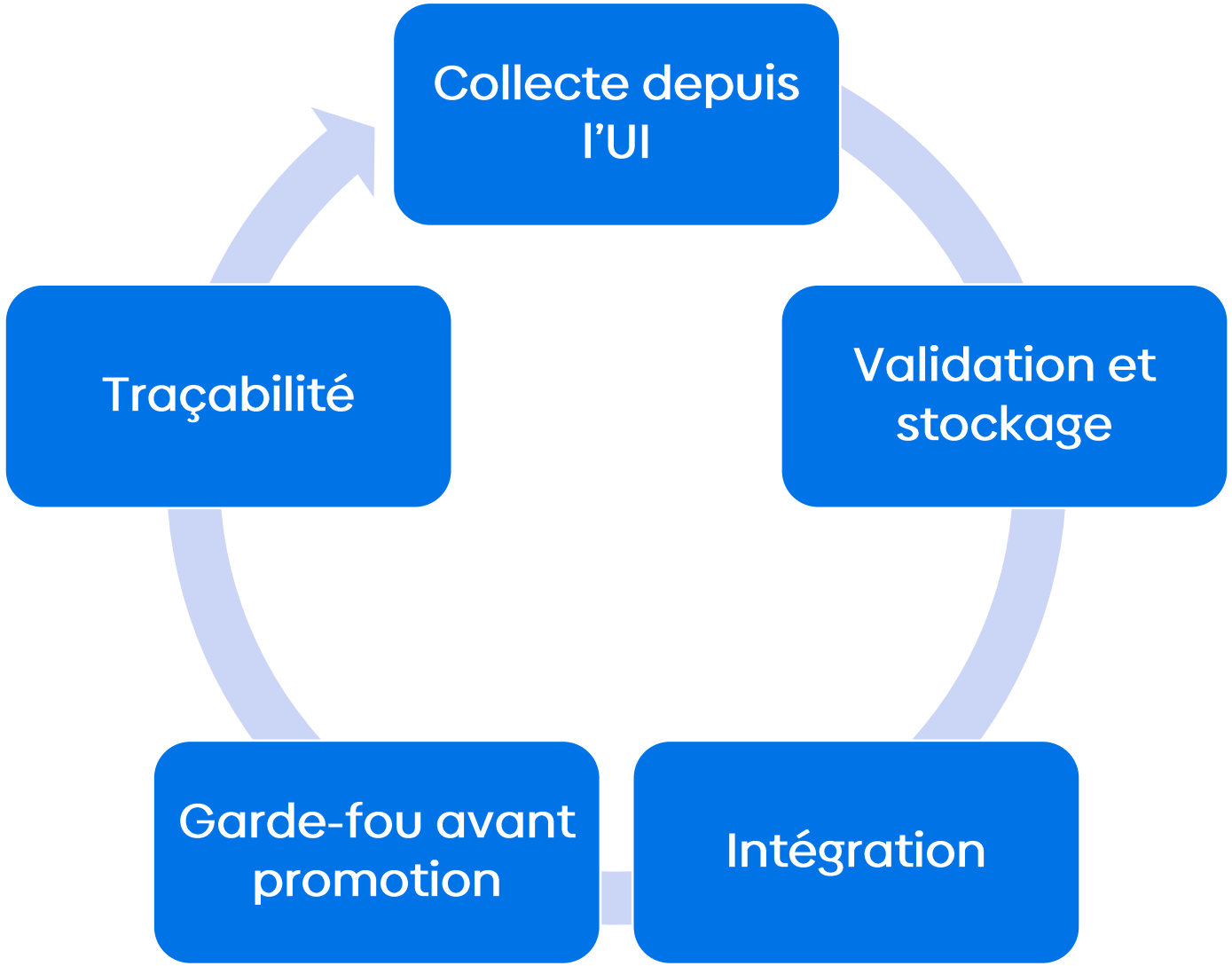
Métriques à surveiller

Quelles métriques surveiller en production ?

Type	Métrique	Seuil d'alerte	Source
Modèle	`f1_macro` de validation (à chaque `POST /train`)	< baseline - 0,02 (tolérance du garde-fou anti-régression déjà implémenté, `DEGRADATION_TOLERANCE` dans `app/main.py`)	Run MLflow (`log_training_run`) comparé à `models/train_metrics_baseline.json`
Modèle	Taux d'abstention (probabilité max < 0,65 ou écart top-2 < 15 %, cf §7.2)	> 20 % sur 4 semaines glissantes	`data/historique_inferences.csv` (à enrichir d'un champ `abstention`, cf §9.4)
Modèle	Erreurs critiques 2→0 (classe réelle « long » prédite « bas »)	> 0 détectée lors d'un audit mensuel des corrections	`data/feedback_conseillers.csv` (corrections conseillers via `POST /feedback`)
Données	PSI (Population Stability Index) sur `age`, `departement`, `code_rome_vise`, `nationalite_hors_ue` et `est_allocataire`	> 0,2	Batch journalier à mettre en place (non implémenté à ce jour, cf §9.3)
Système	Latence p95 API	> 500 ms (cible fixée en §8.7)	`logs/api.log` (middleware `X-Request-ID`), exportable vers Prometheus
Système	Disponibilité	`GET /health` renvoie `status="degraded"` de façon répétée	`HEALTHCHECK` Docker (`docker-compose.yml`/`docker-compose.prod.yml`)
CI/CD	Échec du job `train-validation` sur push vers `main`	Bloque automatiquement `build-and-push` et `deploy`	GitHub Actions (cf §8.6)

Boucle de rétroaction

Comment les retours des conseillers servent-ils à améliorer le modèle ?



- Calendaire
- Volume de nouvelles données
- Performance

06

Conclusion et questions ouvertes